androsec

EXPLOITATION REPORT

AES/ECB mode

owasp.mstg.uncrackable1

Package owasp.mstg.uncrackable1

Severity High

Generated 2026-11-12 17:31 UTC

Disclaimer

This report is an automated assessment and provides an approximate security overview. It is not guaranteed to be 100% accurate. Some vulnerabilities may not have been detected.

Assessment target

Application and device used for this verification.

App name	owasp.mstg.uncrackable1
Package	owasp.mstg.uncrackable1
Version	1.0
Min / Target SDK	19 / 28
APK size	65.1 KB
SHA-256	1da8bf57d266109f9a07c01bf7111a1975ce01f190b9d914bcd3ae3dbef96f21
Device	VirtualBox
Android	9
Serial	10.0.2.5:5555
Report ID	exploitation_20261112_172955
Generated	2026-11-12 17:31 UTC

Executive summary

Finding severity **High** — app risk **MEDIUM** — score **20/100**

0	1	2	0	5
Critical	High	Medium	Low	Info

High AES/ECB mode

RULE	CODE_WEAK_CRYPTO_ECB
CONFIDENCE	High
MASVS	MASVS-CRYPTO-1
EVIDENCE	`AES/ECB/PKCS7Padding`
RECOMMENDATION	Use AES/GCM/NoPadding.

Assessment write-up

Narrative produced for this finding, including scope, methodology, evidence, and remediation.

Executive Summary

This report focuses on the High-severity finding `CODE_WEAK_CRYPTO_ECB` identified in the Android application `owasp.mstg.uncrackable1` v1.0. Static analysis identified the cryptographic string `AES/ECB/PKCS7Padding`, indicating that AES encryption is configured to use Electronic Codebook (ECB) mode.

ECB mode is deterministic: identical 16-byte plaintext blocks encrypt to identical ciphertext blocks. This can reveal patterns in encrypted data without requiring key recovery. The dynamic demonstration was attempted using Frida, but the inline Frida command was rejected with `Error: Invalid or oversized command`. Therefore, this finding is supported by static evidence only in this engagement.

Business impact: if the application encrypts sensitive data with `AES/ECB/PKCS7Padding`, an attacker observing ciphertext may infer repeated data structures and potentially undermine confidentiality. ECB also does not provide authentication or integrity protection.

Scope & Authorization

- Target application: `owasp.mstg.uncrackable1`
- Package: `owasp.mstg.uncrackable1`
- Version: `1.0`
- Target SDK: `28`
- Size: `65.1 KB`
- SHA256: `1da8bf57d266109f9a07c01bf7111a1975ce01f190b9d914bcd3ae3dbef96f21`

Authorization note: This assessment was performed as an authorized university graduation project lab exercise against the intentionally vulnerable OWASP UnCrackable1 benchmark application.

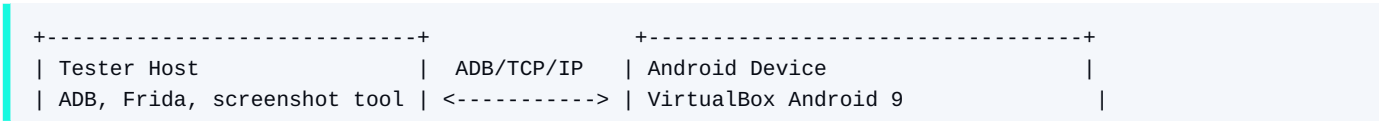
OWASP Mobile Top 10 Classification

OWASP Mobile Top 10: M10 - Insufficient Cryptography

This vulnerability falls under Insufficient Cryptography because the application uses AES in ECB mode with PKCS7 padding. ECB mode is not semantically secure and is considered a weak cryptographic mode under MASVS and OWASP Mobile Top 10. The use of `AES/ECB/PKCS7Padding` violates cryptographic best practices by allowing deterministic ciphertext and pattern leakage.

Environment & Topology

- Tester host: local analysis workstation running ADB, Frida, and screenshot tooling
- ADB transport: TCP/IP to `10.0.2.5:5555`
- Device: VirtualBox Android 9, serial `10.0.2.5:5555`
- Target application: `owasp.mstg.uncrackable1` v1.0



```
|                              |     :5555     | serial=10.0.2.5:5555               |
+------------------------------+                +-----------------------------------+
                                                          |
                                                          v
                                                owasp.mstg.uncrackable1 v1.0
```

Methodology

1. Static analysis identified the cryptographic API pattern `AES/ECB/PKCS7Padding` in the application.
2. Dynamic verification was attempted using Frida to hook `Cipher.getInstance` and demonstrate ECB deterministic weakness.
3. Evidence capture included screenshots before and after exploitation, plus command output.

The full static-to-dynamic verification chain was not completed because the Frida command failed before execution.

Finding Detail

- Rule ID: `CODE_WEAK_CRYPTO_ECB`
- Severity: High
- Confidence: High
- MASVS: `MASVS-CRYPTO-1`
- Static evidence quoted verbatim: `AES/ECB/PKCS7Padding`
- Static recommendation: Use AES/GCM/NoPadding.

Description: The application contains AES encryption configured as `AES/ECB/PKCS7Padding`. ECB mode encrypts each 16-byte plaintext block independently and deterministically. Equal plaintext blocks produce equal ciphertext blocks, leaking data patterns. Additionally, ECB does not provide integrity or authentication.

Exploitation / Verification Steps

Step 1 — Capture baseline UI before exploitation

Command:

```
screenshot
```

Result:

```
Screenshot saved as screenshot_01.png
```

Step 2 — Launch the Uncrackable1 app

Command:

```
adb shell monkey -p owasp.mstg.uncrackable1 -c android.intent.category.LAUNCHER 1
```

Result:

```
bash arg: -p
  bash arg: owasp.mstg.uncrackable1
  bash arg: -c
  bash arg: android.intent.category.LAUNCHER
```

```
  bash arg: 1
Events injected: 1
## Network stats: elapsed time=4ms (0ms mobile, 0ms wifi, 4ms not connected)
```

Step 3 — Attempt Frida hook to demonstrate AES/ECB deterministic weakness

Command attempted:

```
frida -U -n owasp.mstg.uncrackable1 -e "Java.perform(function(){var
Cipher=Java.use('javax.crypto.Cipher');
Cipher.getInstance.overload('java.lang.String').implementation=function(s){console.log('[+]
Cipher.getInstance: '+s); return this.getInstance(s);}; var
SecretKeySpec=Java.use('javax.crypto.spec.SecretKeySpec'); var Arrays=Java.use('java.util.Arrays'); var
key=Java.array('byte',[0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]); var sk=SecretKeySpec.$new(key,'AES');
var c=Cipher.getInstance('AES/ECB/PKCS7Padding'); c.init(1,sk); var block=Java.array('byte',
[65,65,65,65,65,65,65,65,65,65,65,65,65,65,65,65]); var ct1=c.doFinal(block); var ct2=c.doFinal(block);
console.log('[+] ECB block1: '+Arrays.toString(ct1)); console.log('[+] ECB block2:
'+Arrays.toString(ct2)); console.log('[+] Identical ciphertext blocks (ECB deterministic):
'+Arrays.equals(ct1,ct2));});"
```

Result:

```
Error: Invalid or oversized command
```

Assessment: This step failed. No dynamic Frida output was captured, and no runtime demonstration of identical ECB ciphertext blocks was obtained.

Step 4 — Capture final screenshot after ECB vulnerability demonstration

Command:

```
screenshot
```

Result:

```
Screenshot saved as screenshot_04.png
```

Evidence

ARTIFACT	STEP	DESCRIPTION
`screenshot_01.png`	Step 1 — Capture baseline UI before exploitation	Baseline screen captured before app launch and exploitation
Text output	Step 2 — Launch the Uncrackable1 app	Output of `adb shell monkey` launch command
`screenshot_04.png`	Step 4 — Capture final screenshot after ECB vulnerability demonstration	Screen captured after Step 3; Step 3 itself failed with `Error: Invalid or oversized command`

No Frida evidence artifact was collected for Step 3 because the command was rejected before execution.

Remediation

- Replace `AES/ECB/PKCS7Padding` with an authenticated encryption mode, preferably `AES/GCM/NoPadding`.

- Use the following Java/Android configuration:

```
java Cipher.getInstance("AES/GCM/NoPadding");
```

- Generate a unique, non-repeating 12-byte IV/nonce for each encryption operation using `SecureRandom`.
- Store or transmit the IV/nonce alongside the ciphertext and GCM authentication tag.
- Use a 128-bit GCM authentication tag and validate it before decryption.
- Search the codebase for remaining `ECB` or `PKCS7Padding` occurrences and replace them.
- If AES keys are hardcoded, move them to Android Keystore or a securely provisioned backend.
- After remediation, rerun static analysis and perform dynamic verification to confirm that repeated plaintext blocks no longer produce repeated ciphertext blocks.

Conclusion

The High-severity `CODE_WEAK_CRYPTO_ECB` finding remains present based on static evidence. The dynamic exploitation attempt failed because the Frida inline command was too large and was rejected. Residual risk remains High for the affected code until the cryptographic mode is replaced and verified.

Follow-up actions:

- Rerun the Frida demonstration using a script file, for example:

```
bash frida -U -n owasp.mstg.uncrackable1 -l ecb_test.js
```

- Confirm ECB deterministic behavior dynamically, or verify after remediation that no ECB mode is invoked.
- Re-run static analysis and dynamic regression testing after applying the remediation to ensure the High-severity finding is resolved.

Executed steps

Commands that ran and the output captured for each one.

1. Capture baseline UI before exploitation

COMMAND

```
screenshot
```

RESULT

```
Screenshot saved as screenshot_01.png
```

2. Launch the Uncrackable1 app

COMMAND

```
adb shell monkey -p owasp.mstg.uncrackable1 -c android.intent.category.LAUNCHER 1
```

RESULT

```
bash arg: -p
  bash arg: owasp.mstg.uncrackable1
  bash arg: -c
  bash arg: android.intent.category.LAUNCHER
  bash arg: 1
Events injected: 1
## Network stats: elapsed time=4ms (0ms mobile, 0ms wifi, 4ms not connected)
```

3. Attach Frida to the running app, hook Cipher.getInstance, and actively demonstrate AES/ECB weakness by encrypting two identical 16-byte blocks with AES/ECB/PKCS7Padding and logging that the ciphertext blocks are identical

COMMAND

```
frida -U -n owasp.mstg.uncrackable1 -e "Java.perform(function(){var
Cipher=Java.use('javax.crypto.Cipher');
Cipher.getInstance.overload('java.lang.String').implementation=function(s){console.log('[+]
Cipher.getInstance: '+s); return this.getInstance(s);}; var
SecretKeySpec=Java.use('javax.crypto.spec.SecretKeySpec'); var
Arrays=Java.use('java.util.Arrays'); var key=Java.array('byte',
[0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]); var sk=SecretKeySpec.$new(key,'AES'); var
c=Cipher.getInstance('AES/ECB/PKCS7Padding'); c.init(1,sk); var block=Java.array('byte',
[65,65,65,65,65,65,65,65,65,65,65,65,65,65,65,65]); var ct1=c.doFinal(block); var
ct2=c.doFinal(block); console.log('[+] ECB block1: '+Arrays.toString(ct1)); console.log('[+] ECB
block2: '+Arrays.toString(ct2)); console.log('[+] Identical ciphertext blocks (ECB
deterministic): '+Arrays.equals(ct1,ct2));});"
```

RESULT

```
Error: Invalid or oversized command
```

4. Capture final screenshot after ECB vulnerability demonstration

COMMAND

```
screenshot
```

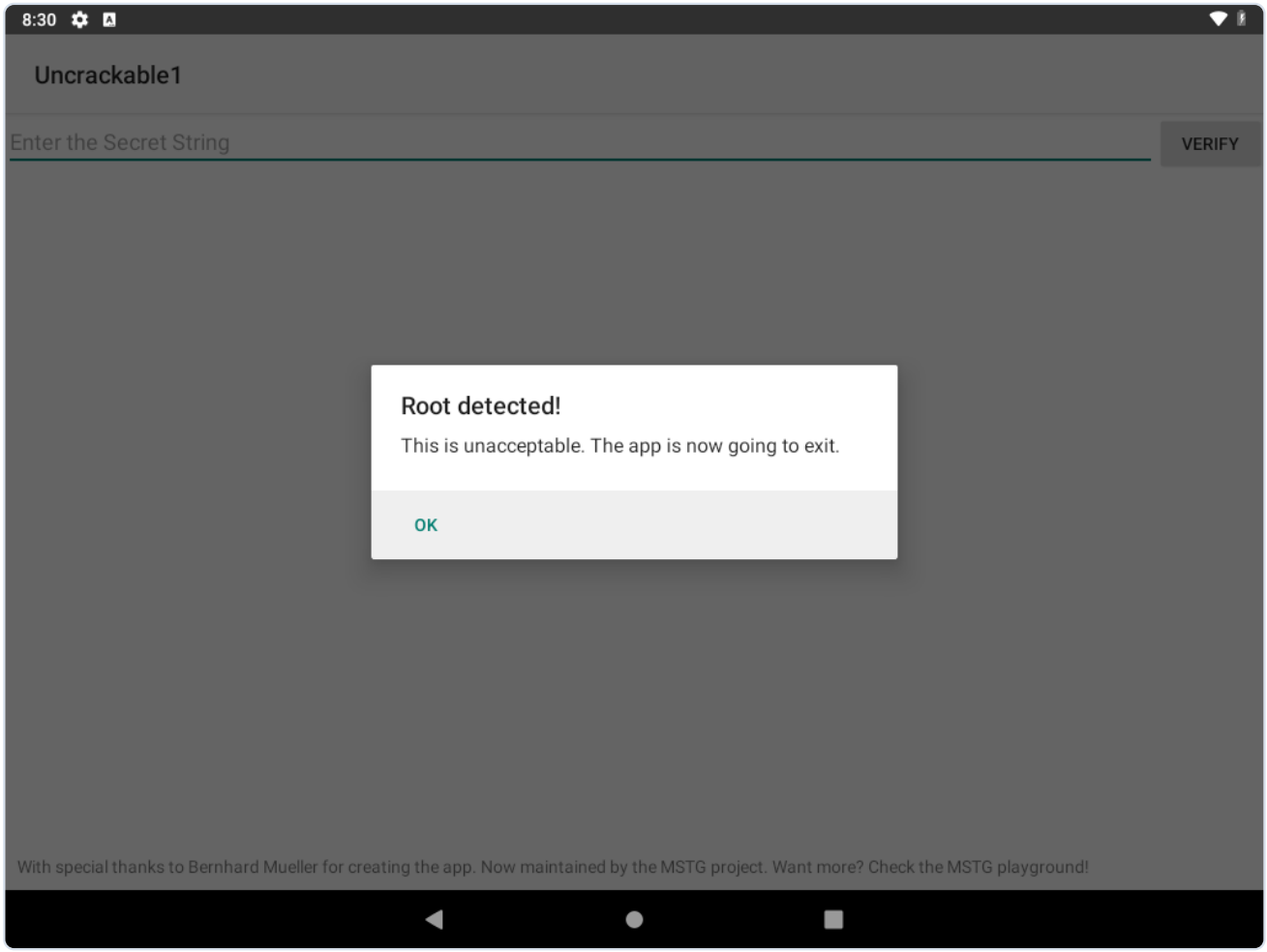
RESULT

```
Screenshot saved as screenshot_04.png
```

Captured evidence

Screenshots and command output saved during the run.

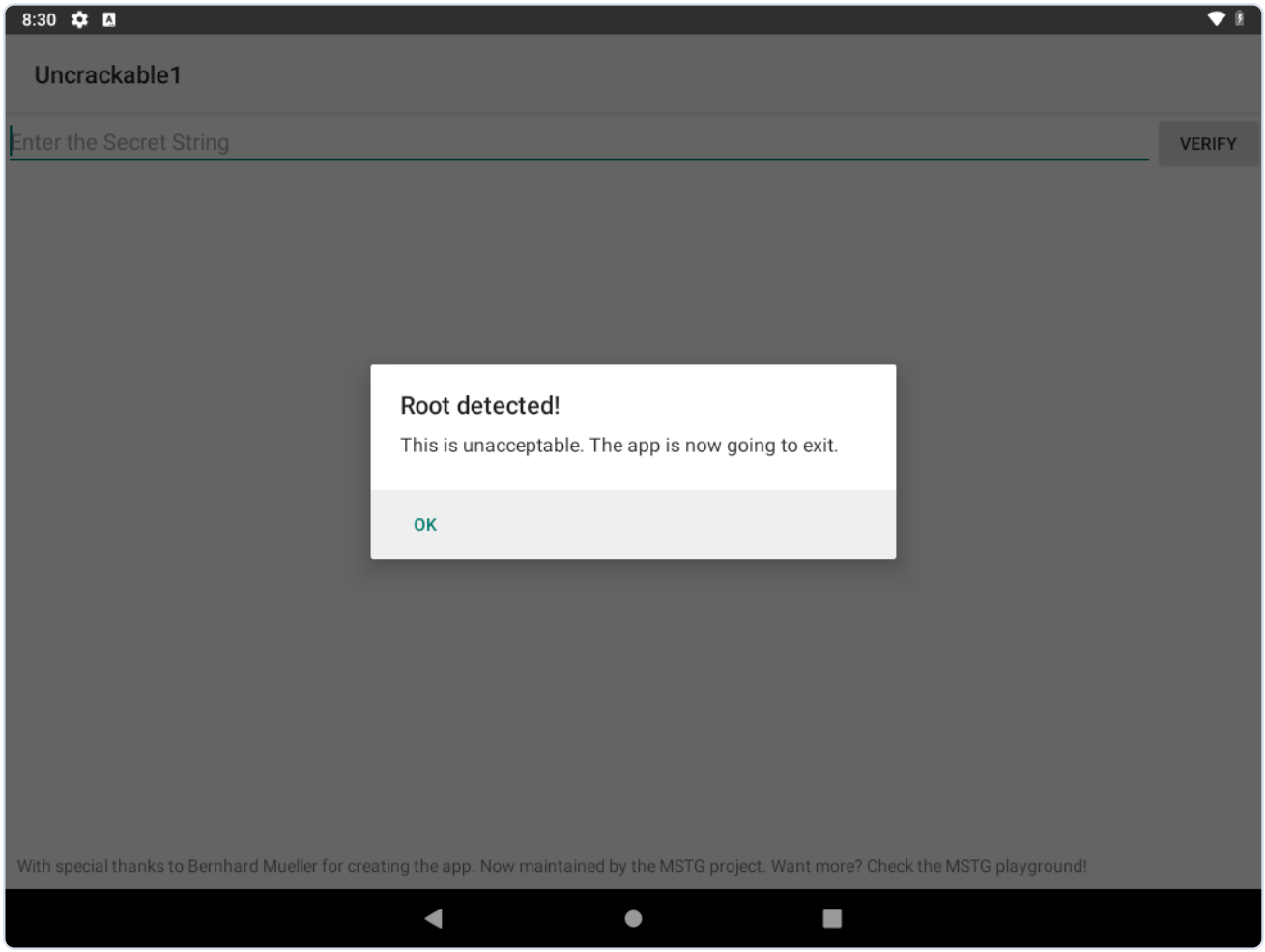
Screenshot — Capture baseline UI before exploitation



Command output — Launch the Uncrackable1 app

```
bash arg: -p
  bash arg: owasp.mstg.uncrackable1
  bash arg: -c
  bash arg: android.intent.category.LAUNCHER
  bash arg: 1
Events injected: 1
## Network stats: elapsed time=4ms (0ms mobile, 0ms wifi, 4ms not connected)
```

Screenshot — Capture final screenshot after ECB vulnerability demonstration



DISCLAIMER

This report is an automated assessment and provides an approximate security overview. It is not guaranteed to be 100% accurate. Some vulnerabilities may not have been detected.